Tutorial - 8

In this tutorial, you will design, debug and execute a few OpenACC programs, and analyze the compiler outputs.

1. Compile and execute the following program. Analyze the output of the *pgcc* compiler. Run the program as ./a.out 10. Analyze the output with *async* clause added to the first parallel region.

```
#include<stdio.h>
#include<stdlib.h>

int main(int argc, char* argv[])
{
   int ngangs = 1;

   if (argc == 2)
      ngangs = atoi(argv[1]);
   else
   {
      printf(" Run the program as ./a.out 10");
      return 1;
   }

   printf("\n ngangs = %d", ngangs);

#pragma acc parallel num_gangs(ngangs)
{
   printf(" Hello world \n");
   printf(" Bye world \n");
}

   printf("Host \n");

#pragma acc parallel num_gangs(ngangs/2)
   printf("Second pragma \n");

   return 0;
}
```

2. Find and fix error(s) in the following program fragment trying to find a number in an integer array. After the code is working, improve parallelism in the code.

```
#pragma  acc parallel
   for (int i = 0; i < N; i++)
      if (numbers[i] == mynumber) break;

   if (i != N) printf("Number not found \n");
   else printf("Number found at location %d \n", i);
```

3. Test the following OpenACC directives using suitable samaple programs:

```
   #pragma acc serial
   num_gangs, num_workers, vector_length
   private, firstprivate, if
```

4. Complete the following programs we have seen in the lecture, compile them using the -Minfo flag and analyze the output for the 3 versions given below.

```
Version 1:
#pragma acc kernels
{
   printf("One \n");
   for (int i = 0 ; i < 10; ++i)
   a[i] = i;
   printf("Two: a[9] = %d \n", a[9]);
   for (int i = 0; i < 10; ++i)
   a[i]* = 2;
   printf("Three: a[9] = %d \n", a[9]);
}
```

```
Version 2:
#pragma acc kernels
{
   printf("One \n");
   for (int i = 0 ; i < 10; ++i)
   {
      a[i] = i;
      printf("first loop %d \n", i);
   }
   printf("Two: a[9] = %d \n", a[9]);
   for (int i = 0; i < 10; ++i)
   {
      a[i]* = 2;
      printf("second loop %d \n", i);
   }
   printf("Three: a[9] = %d \n", a[9]);
}
```

```
Version 3:
#pragma acc kernels
{
   printf("One \n");
   for (int i = 0 ; i < 10; ++i)
   {
      a[i] = i;
      printf("first loop %d \n", i);
   }
   printf("Two \n");
   for (int i = 0; i < 10; ++i)
   {
      a[i]* = 2;
      printf("second loop %d \n", i);
   }
   printf("Three \n");
}
```